Gamma

A Minkowski Spacetime Diagram Generator

Antonio Freixas • June 1, 2022 11:33 AM

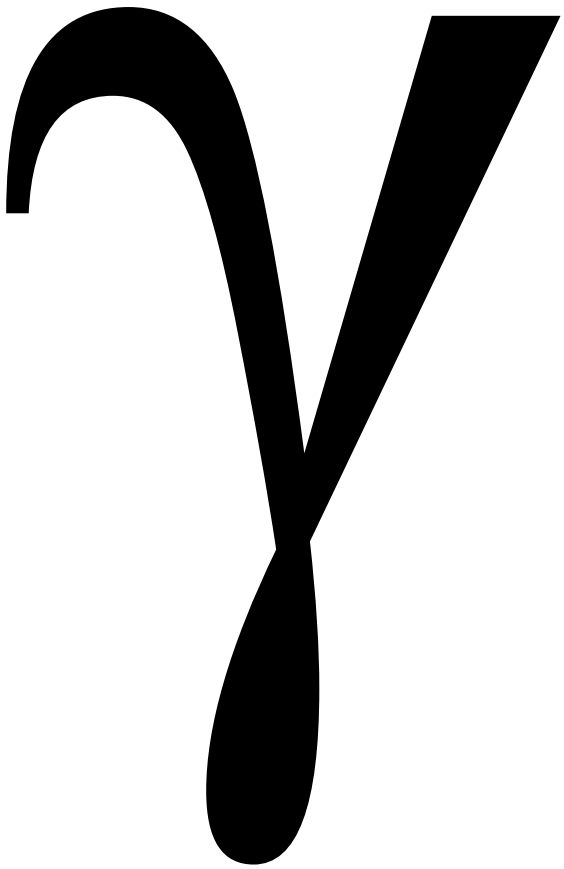

Table of Contents

Introduction

Gamma is a script-based Minkowski spacetime diagramming tool.

Gamma's goals are:

- To provide an easy way to create high-quality spacetime diagrams that can be used to illustrate problems in special relativity.
- To function as a teaching aid for special relativity (experienced physicists create diagrams for people learning special relativity).
- To function as a learning aid for special relativity (students of special relativity create diagrams to further their understanding).
- To solve some simple problems in special relativity.

Scripting

Diagrams are created using a scripting language. There is no way a GUI interface would be able to create the variety of diagrams possible with a script.

Because *Gamma* is scripted, a compilation step is not explicitly required. As long as a script has no syntax errors, it can be immediately executed.

The Default Frame

The script specifies a problem relative to the *default frame* (a default *inertial* frame). In this frame, the velocity is 0 and the origin is $(0, 0)$.

The diagram created from the script, however, can be relative to any frame.

The Basic Elements

Values

Values can be either *primitives* or *objects*. Values can be assigned to *variables*, named entities that can be used in place of whatever value they hold.

Primitives

Gamma's primitives include floating point numbers and strings (sequences of characters). Floating point numbers can also be used to represent integers, angles, colors, and booleans.

Objects

An object is a construct that contains multiple values. The objects available in Gamma simplify the creation of complex diagrams.

An *observer* is an entity that moves in space and time. The set of all points occupied by an observer in spacetime is called a *worldline*.

An observer's worldline is created from multiple segments which can have different velocities and accelerations.

A *frame* is an object representing an inertial frame. It has a velocity and an origin. Frames can be created directly or derived from a point along an observer's worldline.

A *coordinate* is an object that stores two values, x and t. The meaning of the values depends on the context. For all drawing operations, the values represent a location in spacetime using coordinates relative to the resting frame. *Gamma* provides an easy way to transformed coordinates from one frame to another.

A *line* is an infinite straight-line.

A *path* is a sequence of straight line segments. In the future, a path might include other curves.

A *bounds* is a bounding box. It can be used to limit lines.

An *interval* is a range of time, tau, or distance values. It can be used to limit a worldline.

Expressions

Expressions use *operators* to create values. Most operators work on primitives, but some work on objects.

Functions are operators that receive a set of values and return a single new value. The returned value can be of various types, including objects.

Dynamic Variables

Dynamic variables are variables whose values can be changed from outside the script. They are assigned a value only once. This value is used when initially drawing a diagram.

Dynamic variables allow a single script to create many different diagrams, usually variations of a single problem description.

Animation Variables

Animation variables function like any other variable, but their value is determined by the current frame of the animation.

Display Variables

Display variables are variables whose value is controlled by the user through sliders, toggles, or other GUI controls.

Statements

S*tatements* are the basic building blocks of the scripting language, with each statement performing a particular task. Two major statement types are *assignment statements*, which assign a value to a variable, and *commands*, most of which cause something to be drawn.

Commands

Commands guide the creation of a diagram. With them, you can create:

- Axes for any inertial frame
- Grids for any inertial frame
- Hypergrids (a grid of hyperbolas)
- Events (with boosting)
- Lines
- Worldlines
- Paths, open and closed (with fills)
- Labels

Stylesheets

Stylesheets are used to separate a diagram from its presentation. The same problem set up can be represented in various visual styles.

Stylesheets have their own syntax. They can be included within a script or stored in a separate file.

Just about every visual element can be customized.

Other Features

Gamma supports having multiple diagrams displayed at the same time.

Diagrams can be saved as images and animations can be saved as videos.

Diagrams can also be printed.

Script files are edited external to *Gamma*, and so don't require saving. A script is attached to a window; whenever the script is saved, the diagram is updated.

Syntax Notation

The syntax is based on BNF (Backus–Naur Form).

- *Italics* designate non-terminals (symbols that are defined elsewhere and never entered directly).
- **Bold** designates terminals, characters that appear in the script exactly as written. A non-terminal must be used exactly as written and cannot have any internal spaces added (**the** is not equivalent to **t h e**).
- A single non-terminal or terminal is an element.
- Consecutive elements separated by spaces in the syntax can be separated by any amount of whitespace in the script or none at all, unless noted otherwise.
- Consecutive elements that cannot have spaces between them are written inside angled brackets ("<" and ">").
- Parentheses can be used to create elements from groups of terminals and non-terminals.
- Brackets are used to indicate a set of terminal characters (without spaces unless a space is explicitly included in the set) . [**abcd**] is equivalent to <**a** | **b** | **c** | **d**>. A dash is used to indicate all characters within a range. [**a-d**] is also equivalent to <**a** | **b** | **c** | **d**>. If the first character is a circumflex ("^"), then the list represents all characters other than those that follow.
- A vertical bar ("|") signifes a choice: one of the elements.
- An element followed by a question mark ("?") can be used 0 or 1 times.
- An element followed by an asterisk ("*") can be used 0 or more times.
- An element followed by a plus sign ("+") can be used 1 or more times.
- A production begins with a non-terminal, followed by a right arrow ("→"), followed by a sequence of one or more elements and ends with a blank line. Any occurence of the non-terminal on the left can be replaced by the sequence on the right.

A script is syntactically valid if it can be generated starting with the initial non-terminal and following all productions until no non-terminals are left.

Not every syntactically valid script is semantically valid. The semantic rules follow the syntax.

Stylesheet Syntax

stylesheet → *rule**

rule → (*selector* (**,** *selector*)*)? **{** (*styleProperty* **;**)* **}**

selector → <*selectorElement*>+

selectorElement → *command* | *id* | *class*+

command → **display** | **frame** | **animation** | **axes** | **grid** | **hypergrid** | **event** | **line** | **worldline** | **path** | **label**

id → <**#***name*>

class → <**.***name*>

styleProperty → *name* **:** (*float* | *string* | *color* | *name*)

name → [**a-zA-Z_-**][**a-zA-Z_-0-9**]*

color → **#** < (*hexDigit*)3,6,8 > | *colorFunction*

colorFunction → rgb(), hsl() and hsla() formats supported by JavaFX

float → (**+** | **-**) < *digit*+ **.**?> | <*digit****.** *digit*+>)

letter → [**a**-**zA**-**Z**]

digit → [**0**-**9**]

hexDigit → [**0**-**9A**-**F**]

string → **"** [^**"**]* **"** | **'** [^**'**] **'**

Stylesheet Semantics

Overview

Stylesheets are used to separate content from presentation: how the entities are drawn from how they look. By substituting one stylesheet for another, the same data will be drawn differently, while still representing the same underlying model.

Stylesheet properties only control appearance-related attributes like line color, line thickness, tick marks, fonts, etc.

Stylesheets use a different grammar from the main script. The grammar is loosely based on CSS stylesheets, although it is much simpler.

White Space

All consecutive series of whitespace characters (spaces, tabs, carriage returns, and newlines) are treated equivalent to a single space.

Comments

Any character sequence that begins with "/*" and continues up to and including "*/ is removed from the script as long as the "/*" is not inside a string. The effect is as though those characters had not been entered. If the "/*" sequence is outside a string and the next "*/" sequence is inside a string, they will match.

Selectors

selector → <*selectorElement*>+

selectorElement → *command* | *id* | *class*+

command → **display** | **frame** | **animation** | **axes** | **grid** | **hypergrid** | **event** | **line** | **worldline** | **path** | **label**

id → <**#***name*>

class → <**.***name*>

Selectors are used to state to which elements of the script a particular rule applies. There are three types of selectors:

- A *command* selector selects commands with the same name (e.g. "axes" selects "axes" commands).
- An *id* selector selects commands whose id property matches the id selector (e.g. "#id1" matches "axes id: 'id1'). Only one element in a script can have a specific id.
- A *class* selector selects commands whose class property matches the class selector (e.g. ".class1" matches "axis class: 'class1'). The class property can include multiple classes, separated by commas.

Style Properties and Values

styleProperty → *name* **:** (*float* | *string* | *color* | *name*)

The list of style properties is at the end of this section.

There are four types of style property values:

Gamma

- A *float* has the same meaning as for the script. Where the value represents a size, the units are screen units (i.e. pixels). Where the value represents an angle, it is in degrees.
- A *string* has the same syntax as script strings.
- A *color* can be specified in the same ways as with JavaFX's Color.web() function.
- A *name* is valid or invalid depending on the associated style property. Boolean properties accept the names **true** and **false**. Other properties have the acceptable names given in the tables that follow. Colors can also be named. The list of acceptable names are whatever JavaFX's Color.web() function accepts. Names used as values are case-insenstive.

Stylesheet Order

From low to higher precedence:

- All style properties have factory-supplied default values. The values are given in the tables that follow. If there were no stylesheets at all, these values would be used.
- A factory-supplied stylesheet. This is listed below.
- A single stylesheet can be given as a preference. If none is set, it is not used.
- The stylesheets given in the script with the **stylesheet** statement, in the order encountered, with the earlier ones having lower precedence.

The Selection Algorithm

Every command can has a name, and id property (with a single, unique name), a class property (with one or more class names, separated by commas), and a style property which contains only style properties and no selectors.

A selector might look like axes, #id1, .class1, axes#id1, axes.class1, #id.class2, axes#id1.class1.class2, etc. Commands are matched to selectors (and the style properties associated with them) using the following rules:

- Style properties given using the "style" property always have the highest priority. These have no selectors, but always match the command with which they are associated.
- A command matches a selector if it matches every part of a selector. For example, to match axes#id1.class1.class2, the command must be axes , it must have an id of "id1" and two classes named "class1" and "class2". A rule with no selector matches every command.
- Every matching selector is given a score of 1,000 for every id match, 10 for every class name match, and 1 for every command name match. Higher values have higher priority. For matches with the same value, the order encountered determines the precedence, with later matches having higher priority.
- The associated style properties are sorted by precedence.
- Some properties are shorthand for a sequence of others. "color," for example, is a shorthand for a large number of other options. Each shorthand is expanded into its full set.
- Later properties override earlier ones. For example, "textColor: blue; color: red" yields a text color of red, since the expanded color property overrides the textColor setting.
- The final set of properties are what the command will use.

Inherited Values

Some style properties inherit values from others. Inheritance is performed only after the final set of properties have been assembled. Within that list, setting a value for a style property at the top of the inheritance chain sets the value for all the properties that inherit from it. If a lower-level value is assigned first, it will be overwritten.

For example, the **ticks** style property enables tick marks for a set of axes. **xTicks** and **tTicks** inherit from **ticks**. Given this style

```
axes {
  ticks: true;
  xTicks: false;
}
```

xTicks will be disabled, but in this example

```
axes {
  xTicks: false;
  ticks: true;
}
```

xTicks will be enabled.

Style Properties

Here is a complete list of all style properties. The default values are the basic factory defaults. The list is broken down by hierarchies. In the description, the parenthesized items at the end define the types of elements to which that property applies.

The properties actually applied are highlighted in **green**. An element will use the most-specific of the highlighted properties that apply to it. For example, a line's color is specified by **color** since there is nothing more specific that is applicable. But a label's color will come from **textColor** and the x axis's major tick mark's colors will come from **xMajorDivColor**.

Color Hierarchies

Name	Type	Default	Min	Max	Description
color	*color*	#000000			The color for axes, grids, hypergrids, lines, paths, world-lines, event shapes, event boost lines, and text.
x-color	*color*	**color**			The color for the x axis, x grid lines, and x hypergrid lines.
t-color	*color*	**color**			The color for the t axis, t grid lines, and t hypergrid lines.
div-color	*color*	**color**			The color for tick marks, grid lines, and hypergrid lines.
x-div-color	*color*	**div-color**			The color for x tick marks, x grid lines, and x hypergrid lines.
t-div-color	*color*	**div-color**			The color for t tick marks, t grid lines, and t hypergrid lines.
major-div-color	*color*	**div-color**			The color for major tick marks, and major grid lines.
x-major-div–color	*color*	**major-div–color**			The color for major x tick marks, and major x grid lines.
t-major-div–color	*color*	**major-div–color**			The color for major t tick marks, and major t grid lines.
text-color	*color*	**color**			The color for all text.
x-text-color	*color*	**text-color**			The color for x tick marks.
t-text-color	*color*	**text-color**			The color for t tick marks.
background–color	*color*	#FFFFFF			The diagram background color and the fill color.

Line Hierarchies

Name	Type	Default	Min	Max	Description
line-thickness	*float*	1.0	> 0		The line thickness for axes, tick marks, grids, hypergrids, lines, worldlines, and event boost lines.
x-line-thickness	*float*	**line-thickness**	> 0		The line thickness for x axes, x tick marks, x grid lines, and x hypergrids lines.

Gamma

Name	Type	Default	Min	Max	Description
t-line-thickness	*float*	line-thickness	> 0		The line thickness for t axes, t tick marks, t grid lines, and t hypergrids lines.
div-line-thickness	*float*	line-thickness	> 0		The line thickness for tick marks, x grid lines, and x hypergrid lines.
x-div-line-thickness	*float*	div-line-thickness	> 0		The line thickness for x tick marks, x grid lines, and x hypergrid lines.
t-div-line-thickness	*float*	div-line-thickness	> 0		The line thickness for t tick marks, t grid lines, and t hypergrid lines.
major-div-line-thickness	*float*	div-line-thickness	> 0		The line thickness for major tick marks, and major grid lines.
x-major-div-line-thickness	*float*	major-div-line-thickness	> 0		The line thickness for major x tick marks, and major x grid lines.
t-major-div-line-thickness	*float*	major-div-line-thickness	> 0		The color for major t tick marks, and major t grid lines.
line-style	**solid**, **dashed**, **dotted**	**solid**			The line style for axes, tick marks, grids, hypergrids, lines, worldlines, and event boost lines.
x-line-style	**solid**, **dashed**, **dotted**	**line-style**			The line style for x axes, x tick marks, x grid lines, and x hypergrids lines.
t-line-style	**solid**, **dashed**, **dotted**	**line-style**			The line style for t axes, t tick marks, t grid lines, and t hypergrids lines.
div-line-style	**solid**, **dashed**, **dotted**	**line-style**			The line style for tick marks, x grid lines, and x hyper-grid lines.
x-div-line-style	**solid**, **dashed**, **dotted**	**div-line-style**			The line style for x tick marks, x grid lines, and x hyper-grid lines.
t-div-line-style	**solid**, **dashed**, **dotted**	**div-line-style**			The line style for t tick marks, t grid lines, and t hyper-grid lines.
major-div-line-style	**solid**, **dashed**, **dotted**	**div-line-style**			The line style for major tick marks, and major grid lines.
x-major-div-line-style	**solid**, **dashed**, **dotted**	**major-div-line-style**			The line style for major x tick marks, and major x grid lines.
t-major-div-line-style	**solid**, **dashed**, **dotted**	**major-div-line-style**			The color for major t tick marks, and major t grid lines.

Font Hierarchies

Name	Type	Default	Min	Max	Description
font-family	*string*	System dependent			Set the font family for any text.
tick-font-family	*string*	**font-family**			Set the font family for tick labels.
x-tick-font-family	*string*	**tick-font-family**			Set the font family for x tick labels.
t-tick-font-family	*string*	**tick-font-family**			Set the font family for t tick label.
font-weight	**normal**, others[1]	**normal**			Set the font weight for any text.
tick-font-weight	**normal**, others[1]	**font-weight**			Set the font weight for tick labels.
x-tick-font-weight	**normal**, others[1]	**tick-font-weight**			Set the font weight x tick labels.
x-tick-font-weight	**normal**, others[1]	**tick-font-weight**			Set the font weight t tick labels.
font-style	**regular**, **italic**	**regular**			Set the font style for any text.
tick-font-style	**regular**, **italic**	**font-style**			Set the font style for tick labels.
x-tick-font-style	**regular**, **italic**	**tick-font-style**			Set the font style for x tick labels.

Name	Type	Default	Min	Max	Description
t-tick-font-style	**regular**, **italic**	**tick-font-style**			Set the font style for t tick labels.
font-size	*float*	10.0	> 0		Set the font size for any text. This is the height of the text from ascender to descender. (text)
tick-font-size	*float*	**font-size**	> 0		Set the font size for tick labels.
x-tick-font-size	*float*	**tick-font-size**	> 0		Set the font size for x tick labels.
t-tick-font-size	*float*	**tick-font-size**	> 0		Set the font size for t tick labels.

[1] Other valid font weights are: **thin**, **extra light**, **ultra light**, **light**, **medium**, **semi bold**, **demi bold**, **bold**, **extra bold**, **ultra bold**, **black**, **heavy**, and numbers 0 to 1000 (with the latter being heaviest). Weights with spaces in their name must be quoted.

Text Hierarchies

Name	Type	Default	Min	Max	Description
text-padding	*float*	2.0	0		Set the text padding. Find the bounding box of the text (from left to right and from highest ascender to lowest descender) and pad each side of the box with the given number of transparent pixels.
text-padding-top	*float*	**text-padding**	0		Set the top text padding.
text-padding–bottom	*float*	**text-padding**	0		Set the bottom text padding.
text-padding-left	*float*	**text-padding**	0		Set the top left padding.
text-padding-right	*float*	**text-padding**	0		Set the top right padding.
text-anchor	**TL**, **TC**, **TR**, **ML**, **MC**, **MR**, **BL**, **BC**, **BR**	**TC**			Calculate the bounding box for a text string, including the padding. Determine the 9 possible anchor points. T=top, M=middle, B=baseline, L=left, C=center, R=right. The anchor point is the point that is positioned at the given text position.

Tick Hierarchies

Name	Type	Default	Min	Max	Description
ticks	*boolean*	**true**			Draw the tick marks.
x-ticks	*boolean*	**ticks**			Draw the x tick marks.
t-ticks	*boolean*	**ticks**			Draw the t tick marks.
tick-labels	*boolean*	true			Label the tick marks (only if drawn).
x-tick-labels	*boolean*	**tick-labels**			Label the x tick marks (only if drawn).
t-tick-labels	*boolean*	**tick-labels**			Label the t tick marks (only if drawn).
tick-length	*float*	3.0	0		The length of the tick marks. The actual length is this length + the axis line thickness.
major-tick-length	*float*	10.0	0		The length of the major tick marks. The actual length is this length + the axis tick line thickness.

Hypergrid

Name	Type	Default	Min	Max	Description
left-quadrant	*boolean*	**true**			Draw the hypergrid in the left quadrant.
right-quadrant	*boolean*	**true**			Draw the hypergrid in the right quadrant.
top-quadrant	*boolean*	**true**			Draw the hypergrid in the top quadrant.
bottom-quadrant	*boolean*	**true**			Draw the hypergrid in the bottom quadrant.

Gamma

Miscellaneous

Name	Type	Default	Min	Max	Description
opacity	*double*	1.0	0.0	1.0	Sets the opacity of all elements generated by a command. 0.0 is completely transparent; 1.0 is completely opaque.
arrow	**none**, **both**, **start**, **end**	**, none**			Place arrowheads at the start, end, or both of lines. This applies to the explicitly clipped ends of lines or world-lines, to non-closed paths, to axes, and to event boost lines. For most things, the start is the end with the smaller t coordinate. If the t coordinates are the same for both ends, the start is the end with the smaller x coordinate. For paths, the start and end are defined by the path.
arrow-width	*float*	10.0	>0		If the arrowhead is horizontal, this is its width.
arrow-height	*float*	8.0	>0		If the arrowhead is horizontal, this is its height.
event-diameter	*float*	5.0	> 0		Set the diameter of an event shape.
event-shape	**circle**, **square**, **diamond**, **star**	**circle**			The shape representing an event. A circle that bounds the shape determines the diameter. The center of the circle is the shape's attachment point.

Color Names

The following color names may be used wherever a color is required. The corresponding hex value is in parentheses:

ALICEBLUE:	(#F0F8FF)	ANTIQUEWHITE:	(#FAEBD7)	AQUA:	(#00FFFF)
AQUAMARINE:	(#7FFFD4)	AZURE:	(#F0FFFF)	BEIGE:	(#F5F5DC)
BISQUE:	(#FFE4C4)	BLACK:	(#000000)	BLANCHEDALMOND:	(#FFEBCD)
BLUE:	(#0000FF)	BLUEVIOLET:	(#8A2BE2)	BROWN:	(#A52A2A)
BURLYWOOD:	(#DEB887)	CADETBLUE:	(#5F9EA0)	CHARTREUSE:	(#7FFF00)
CHOCOLATE:	(#D2691E)	CORAL:	(#FF7F50)	CORNFLOWERBLUE:	(#6495ED)
CORNSILK:	(#FFF8DC)	CRIMSON:	(#DC143C)	CYAN:	(#00FFFF)
DARKBLUE:	(#00008B)	DARKCYAN:	(#008B8B)	DARKGOLDENROD:	(#B8860B)
DARKGRAY:	(#A9A9A9)	DARKGREEN:	(#006400)	DARKGREY:	(#A9A9A9)
DARKKHAKI:	(#BDB76B)	DARKMAGENTA:	(#8B008B)	DARKOLIVEGREEN:	(#556B2F)
DARKORANGE:	(#FF8C00)	DARKORCHID:	(#9932CC)	DARKRED:	(#8B0000)
DARKSALMON:	(#E9967A)	DARKSEAGREEN:	(#8FBC8F)	DARKSLATEBLUE:	(#483D8B)
DARKSLATEGRAY:	(#2F4F4F)	DARKSLATEGREY:	(#2F4F4F)	DARKTURQUOISE:	(#00CED1)
DARKVIOLET:	(#9400D3)	DEEPPINK:	(#FF1493)	DEEPSKYBLUE:	(#00BFFF)
DIMGRAY:	(#696969)	DIMGREY:	(#696969)	DODGERBLUE:	(#1E90FF)
FIREBRICK:	(#B22222)	FLORALWHITE:	(#FFFAF0)	FORESTGREEN:	(#228B22)
FUCHSIA:	(#FF00FF)	GAINSBORO:	(#DCDCDC)	GHOSTWHITE:	(#F8F8FF)
GOLD:	(#FFD700)	GOLDENROD:	(#DAA520)	GRAY:	(#808080)
GREEN:	(#008000)	GREENYELLOW:	(#ADFF2F)	GREY:	(#808080)
HONEYDEW:	(#F0FFF0)	HOTPINK:	(#FF69B4)	INDIANRED:	(#CD5C5C)
INDIGO:	(#4B0082)	IVORY:	(#FFFFF0)	KHAKI:	(#F0E68C)
LAVENDER:	(#E6E6FA)	LAVENDERBLUSH:	(#FFF0F5)	LAWNGREEN:	(#7CFC00)
LEMONCHIFFON:	(#FFFACD)	LIGHTBLUE:	(#ADD8E6)	LIGHTCORAL:	(#F08080)
LIGHTCYAN:	(#E0FFFF)	LIGHTGOLDENRODYELLOW:	(#FAFAD2)	LIGHTGRAY:	(#D3D3D3)
LIGHTGREEN:	(#90EE90)	LIGHTGREY:	(#D3D3D3)	LIGHTPINK:	(#FFB6C1)
LIGHTSALMON:	(#FFA07A)	LIGHTSEAGREEN:	(#20B2AA)	LIGHTSKYBLUE:	(#87CEFA)
LIGHTSLATEGRAY:	(#778899)	LIGHTSLATEGREY:	(#778899)	LIGHTSTEELBLUE:	(#B0C4DE)
LIGHTYELLOW:	(#FFFFE0)	LIME:	(#00FF00)	LIMEGREEN:	(#32CD32)
LINEN:	(#FAF0E6)	MAGENTA:	(#FF00FF)	MAROON:	(#800000)
MEDIUMAQUAMARINE:	(#66CDAA)	MEDIUMBLUE:	(#0000CD)	MEDIUMORCHID:	(#BA55D3)
MEDIUMPURPLE:	(#9370DB)	MEDIUMSEAGREEN:	(#3CB371)	MEDIUMSLATEBLUE:	(#7B68EE)
MEDIUMSPRINGGREEN:	(#00FA9A)	MEDIUMTURQUOISE:	(#48D1CC)	MEDIUMVIOLETRED:	(#C71585)
MIDNIGHTBLUE:	(#191970)	MINTCREAM:	(#F5FFFA)	MISTYROSE:	(#FFE4E1)
MOCCASIN:	(#FFE4B5)	NAVAJOWHITE:	(#FFDEAD)	NAVY:	(#000080)
OLDLACE:	(#FDF5E6)	OLIVE:	(#808000)	OLIVEDRAB:	(#6B8E23)
ORANGE:	(#FFA500)	ORANGERED:	(#FF4500)	ORCHID:	(#DA70D6)
PALEGOLDENROD:	(#EEE8AA)	PALEGREEN:	(#98FB98)	PALETURQUOISE:	(#AFEEEE)
PALEVIOLETRED:	(#DB7093)	PAPAYAWHIP:	(#FFEFD5)	PEACHPUFF:	(#FFDAB9)
PERU:	(#CD853F)	PINK:	(#FFC0CB)	PLUM:	(#DDA0DD)
POWDERBLUE:	(#B0E0E6)	PURPLE:	(#800080)	RED:	(#FF0000)

ROSYBROWN:	(#BC8F8F)	ROYALBLUE:	(#4169E1)	SADDLEBROWN:	(#8B4513)
SALMON:	(#FA8072)	SANDYBROWN:	(#F4A460)	SEAGREEN:	(#2E8B57)
SEASHELL:	(#FFF5EE)	SIENNA:	(#A0522D)	SILVER:	(#C0C0C0)
SKYBLUE:	(#87CEEB)	SLATEBLUE:	(#6A5ACD)	SLATEGRAY:	(#708090)
SLATEGREY:	(#708090)	SNOW:	(#FFFAFA)	SPRINGGREEN:	(#00FF7F)
STEELBLUE:	(#4682B4)	TAN:	(#D2B48C)	TEAL:	(#008080)
THISTLE:	(#D8BFD8)	TOMATO:	(#FF6347)	TRANSPARENT:	(#00000000.)
TURQUOISE:	(#40E0D0)	VIOLET:	(#EE82EE)	WHEAT:	(#F5DEB3)
WHITE:	(#FFFFFF)	WHITESMOKE:	(#F5F5F5)	YELLOW:	(#FFFF00)
YELLOWGREEN:	(#9ACD32)				

Color Functions

There are four color functions:

- **rgb()**: This takes three values, for red, green, and blue. The values can range from 0 to 255 or 0 to 100%.
- **rgba()**: This takes four values. The first three are the same as for rgb(). The fourth is an alpha value from 0.0 to 1.0.
- **hls()**: This takes three values. The first is a hue, from 0 to 360, with red being 0 (or 360). The second values range from 0 to 100% and are saturation and lightness.
- **hsla()**: This takes four values. The first three are the same as for hsl(). The fourth is an alpha value from 0.0 to 1.0.

Default Stylesheet

This is the default stylesheet. Since most style properties default to the values given above, the default stylesheet is mainly useful for command-specific defaults.

```
grid { color: #CCC; }
hypergrid { color: #3D3; }
worldline { color: #00F; }
axes { font-size: 14; tick-font-size: 10; }
```

Script Syntax

program → *statementBlock* | *slideshow*

slideshow → **slideshow** *slideshowSettings*? *slideshowBlock*

slideshowSettings → *slideshowSetting* (**,** *slideshowSetting*)*

slideshowSetting → **autoPlay** | **defaultPause** *float*

slideshowBlock → **{** *scriptStatement* (**;** *scriptStatement*)* **}**

scriptStatement → **script** *string scriptSettings*?

scriptSettings → *scriptSetting* (**,** *scriptSetting*)*

scriptSetting → **pause** *float*

statementBlock → (*statement* | **{** *statementBlock* **}**)*

statement → *ifStatement* | *whileStatement* | *forStatement* | **{** *statementBlock* **}** | *semicolonStatement*

ifStatement → **if (** *expr* **)** *statement* (**else** *statement*)?

whileStatement → **while (** *expr* **)** *statement*

forStatement → **for** *variable* **=** *expr* **to** *expr* **step** *expr statement*

semicolonStatement → *simpleStatement*? **;**

simpleStatement →
includeStatement | *stylesheetStatement* | *setStatement* | *printStatement* | *assignmentStatement* | *commandStatement* | *breakStatement* | *continueStatement*

includeStatement → **include** *string*

stylesheetStatement → **stylesheet external*** *string*

setStatement → **set** *setting* (**,** *setting*)*

setting → **units :** *float* | **displayPrecision :** *float* | **printPrecision :** *float* | **minVersion :** *string*

printStatement → **print** *expr?*

assignmentStatement →
(*leftVariable* **=** *expr*) |
(**static** *leftVariable* **=** *expr*) |
(**animate** *leftVariable* **=** *expr* (**to** *expr*)? **step** *expr*) |
(**range** *leftVariable* **=** *expr* **from** *expr* **to** *expr* **label** *expr*) |
(**toggle** *leftVariable* **=** *expr* **label** *expr* **restart**?) |
(**choice** *leftVariable* **=** *expr* **choices** *expr* (**,** *expr*)* **label** *expr* **restart**?)

leftVariable → *name* | *leftVariable* **.** *name*

commandStatement → *command propertyList*?

command →
display | **frame** *expr?* | **animation** | **axes** *expr?* | **grid** *expr?* | **hypergrid** | **event** *expr* | **line** *expr* |
worldline *expr* | **path** *expr* | **label** *expr*

propertyList → *propertyElement,* (**,** *propertyElement*)*

propertyElement → *property* | *expr*

property → *name* **:** *expr*

continueStatement → **continue**

breakStatement → **break**

Gamma

object → **[** (*observer* | *frame* | *line* | *path* | *bounds* | *interval*) **]** | *coord*

observer → **observer** *worldlineList?*

worldlineList → *worldlineInitializer* | *worldlineSegmentList* | (*worldlineInitializer* **,** *wordlineSegmentList*)

worldlineInitializer → (**origin** *expr* | **distance** *expr* | **tau** *expr*)*

worldlineSegmentList → *worldlineSegment* (**,** *worldlineSegment*)*

worldlineSegment → (**velocity** *expr* | **acceleration** *expr* | **velocity** *expr* **acceleration** *expr*) *segmentLimit?*

segmentLimit → **time** *expr* | **tau** *expr* | **distance** *expr* | **velocity** *expr*

frame →
(**frame observer** *expr* (**at** (**time** *expr* | **tau** *expr* | **distance** *expr* | **velocity** *expr*))?) |
(**frame** ((**origin** *expr*) | (**velocity** *expr*))+)

line → **line** (
(**axis** (**x** | **t**) *expr* (**offset** *expr)?*) |
(**angle** *expr* **through** *expr*) |
(**from** *expr* **to** *expr*)
)

path → *expr* (**,** *expr*)+

bounds → **bounds** *expr expr*

interval → **interval** (**time** | **tau** | **distance**) *expr* **to** *expr*

coord → **(** *expr* **,** *expr* **)**

expr → *or* | (*expr* (**<-** | **->**) *or*)

or → *and* | (*expr* **||** *and*)

and → *equality* | (*expr* **&&** *equality*)

equality → *comparison* | (*expr* (**==** | **!=**) *comparison*)

comparison → *sum* | (*expr* (**<** | **>** | **<=** | **>=**) *sum*)

sum → *mult* | (*expr* (**+** | **-**) *mult*)

mult → *exp* | (*mult* (***** | **/** | **%**) *exp*)

exp → *unary* | (*unary* **^** *exp*)

unary → *postfix* | (**!** | **+** | **-**) *postfix*

postfix → *primary* | *postfix* **.** *name*

primary → *element* | **(** *expr* **)**

element → *float* | *variable* | *function* | *object*

function → *name* **(** *argList* **)**

argList → *expr* (**,** *expr*)*

variable → *name* (**[** *expr* **]**)?

name → < *letter* | _ >< *letter* | *digit* | _ >*

boolean → *float*

float → < *digit*+ **.**?> | <*digit****.** *digit*+>)

letter → [**a**-**zA**-**Z**]

digit → [**0**-**9**]

string → " [^"]* " | ' [^'] '

Script Semantics

White Space

All consecutive series of whitespace characters (spaces, tabs, carriage returns, and newlines) are treated equivalent to a single space.

Comments

Any character sequence that begins with "//" and continues up to but not including a newline is removed from the script. The effect is as though those characters had not been entered.

Any character sequence that begins with "/*" and continues up to and including "*/ is removed from the script as long as the "/*" is not inside a string. The effect is as though those characters had not been entered. If the "/*" sequence is outside a string and the next "*/" sequence is inside a string, they will match.

Values

Gamma supports several different kinds of values: floats (floating point or *real* numbers), strings, observers, frames, lines, paths, coordinates, and property lists. A value can be written literally or stored in a named association called a *variable*.

There are a few value subtypes: cos, colors, booleans, and degrees. When these are required, a float value is used.

When a specific type of value is required and a different type is given, an error is reported but with some exceptions:

- When one of the subtypes is required, a float is used but the value is given this reinterpretation:
  - A floating point number is converted to an integer using the round half away from zero method (23.5 → 24, -23.5 → -24).
  - A floating point number is converted to a color by first converting it to an integer and masking with 0xFFFFFFFF. The four bytes left are assigned to red, green, blue, and alpha, respectively, starting with the highest ordered byte.
  - A floating point number is converted to a boolean by checking whether it is 0 (false) or not 0 (true).
  - A degree is normalized to be from 0 to 360. 0° means horizontal. Anchored at (0, 0), a 0° line would be drawn to the left (3 o'clock position) while a 180° line would be drawn to the right (9 o'clock position).
- When a string is required and any value given, the value is converted to its character representation (i.e. the floating point value 10 becomes the character string "10").
- When a frame object is required, an observer object can be given. [observer a] becomes [frame observer [observer a] at tau 0].

Primitives and Objects

A primitive is an entity with a single value. An object is an entity comprised of multiple values.

Floats and strings are primitives. Observers, frames, lines, paths, property lists, intervals, and coordinates are objects.

Literals

For primitives, a literal is a single element whose value is apparent from viewing. The element *10* is a floating point literal whose value is ten. The element "*abc*" is a string literal whose value is the sequence of characters "a", "b", and "c".

For objects, a literal is a sequence of characters that create the object. For all objects except coordinates, the sequence begins with "[", is followed by the object type, and ends with "]". Within the brackets, variables can be used, so the value of the object literal may depend on other parts of the script.

Coordinate literals begin with "(" and end with ")". The type name is not included.

In the following sections, the term *expr* should evaluate to a numeric expression unless it is followed by another type name in parentheses. For example, *expr(string)* must evaluate to a string and *expr(frame)* must evaluate to a frame.

Floats

(**+** | **-**) (*digit*+ **.**? | *digit** **.** *digit*+)

digit → [**0**-**9**]

Floating point numbers can be written in normal decimal point form, but not in scientific/engineering notation. This form can be also used to write the integer and boolean value subtypes. For that matter, although it is clumsy, it can also be used to write color values, although color literals are easier to use.

Strings

" [^**"**]* **"** | **'** [^**'**] **'**

Strings are sequences of characters surrounded by the same quoting character, either single-quote (**'**) or double-quote (**"**). The following character pairs can be used to include special characters into the string:

- \" – insert a double-quote
- \' – insert a single-quote
- \n – insert a newline
- \\ – insert a backslash

Observers

observer → **observer** *worldlineList?*

worldlineList → *worldlineInitializer* | *worldlineSegmentList* | (*worldlineInitializer* **,** *wordlineSegmentList*)

worldlineInitializer → (**origin** *expr* | **distance** *expr* | **tau** *expr*)*

worldlineSegmentList → *worldlineSegment* (**,** *worldlineSegment*)*

worldlineSegment → (**velocity** *expr* | **acceleration** *expr* | **velocity** *expr* **acceleration** *expr*) *segmentLimit?*

segmentLimit → **time** *expr* | **tau** *expr* | **distance** *expr* | **velocity** *expr*

Observers are entities that move in space and time. Their path through spacetime is called a worldline.

First, some terminology:

- x refers to a position relative to the origin in the default frame.
- t refers to a time relative to the origin in the default frame.
- d refers to the distance traveled on a worldline since the worldline's origin. It is measured as a length relative to the default frame.
- *tau* refers to the time on a worldline since the worldline's origin.
- v refers to the velocity of the worldline at a given point.

A worldline is defined using a sequence of connected segments. Segments include their first point, but not their last.

Every point on a worldline has an associated set of scalar *x*, *t*, *d*, *tau*, and *v* values.

The worldline is initialized by optionally specifying the initial *x*, *t*, *d*, and *tau* values for the first segment (the origin provides the starting *x* and *t* values). If an initializer appears more than once, an error occurs.

If the origin *(x, t)* is not specified, it is *(0, 0)*.

If *d* is not specified, it is 0.

If *tau* is not specified, it is 0.

All other segments begin with the *x*, *t*, *d*, and *tau* values that the prior segment ends with.

Each segment has a starting velocity, specified in fractions of *c*, and a constant acceleration (which could be 0), specified in *g*'s.

- If no velocity is give, the starting velocity of a segment is equal to the ending velocity of the prior segment. If it is the first segment, the velocity is 0.
- If a velocity is given, then that velocity is used. This can produce and instantaneous acceleration between segments, which is unphysical, but useful in diagramming certain problems.
- If no acceleration is given, it is set to 0. Acceleration is not inherited from the prior segment.

Each segment can have a limit, which is a delta *t*, *tau*, or *d* from the start of the segment to the end. This value must be greater than 0. It can also be an absolute velocity, which can be any floating point value. A segment without a limit continues forever and cannot be followed by another segment.

Worldline's are infinite. The first segment extends backwards for an infinite time. If the last segment has a limit, it is ignored; the segment extends forwards for an infinite time.

Examples

The simplest observer is:

```
[observer]
```

This is an observer with one defined segment whose velocity is 0, acceleration is 0, and begins at position (0, 0) with a *tau* of 0 and a *d* of 0. This segment continues infinitely from there. There is a second implied segment prior to this one, extending infinitely in the opposite direction at velocity 0.

```
[observer velocity 0.8]
```

This is an observer with one defined segment whose velocity is 0.8 *c*, acceleration is 0, and begins at position (0, 0) with a *tau* of 0 and a *d* of 0. This segment continues infinitely from there. There is a second implied segment prior to this one, extending infinitely in the opposite direction at velocity 0.8 *c*.

```
[observer acceleration 0.1 distance 5]
```

This is an observer with one defined segment whose velocity starts at 0 with an acceleration of 0.1 g's. The segment begins at position (0, 0) with a *tau* of 0 and a *d* of 0. This segment continues until *d* = 5. There is a second implied segment prior to this one, extending infinitely in the opposite direction at velocity 0. There is a third implied segment after this one, extending infinitely in the same direction at whatever velocity the first segment ended at.

```
[velocity 0.8 distance 4, velocity -0.8 distance 4]
```

This is a worldline that can be used to illustrate the twins paradox. The segment begins at position (0, 0) with a *tau* of 0 and a *d* of 0. The observer starts with velocity 0.8 *c* for (typically) 4 light years, makes an instantaneous turn and travels back the 4 light years. The setup implies a 0.8 *c* velocity prior to the defined journey and a -0.8 *c* velocity after it. Usually, the traveling twin first passes the stationary twin when *x* = *t* = *tau* = 0, which is the case here. The defined journey totals 8 light years, so *d* = 8 at the end of the second segment.

Frames

frame →
(**frame observer** *expr* (**at** (**time** *expr* | **tau** *expr* | **distance** *expr* | **velocity** *expr*))?) |
(**frame** ((**velocity** *expr*) | (**origin** *expr*))*)

Gamma

A frame refers to an inertial frame, which has a constant velocity and origin. In physics, relative velocity is sufficient to identify an inertial frame; for example, two observers some distance apart but who are motionless relative to each other are considered to be in the same inertial frame. But since we will use frames to define a coordinate system, two frames with the same velocity and different origins are not the same.

A frame can be derived from an observer and a point along their worldline. This corresponds to the first form given above, with the first expression evaluating to an observer. If the **at** clause is omitted, the frame is the instantaneous moving frame where the worldline's *tau = 0*. If the **at** clause is included, then it is the instantaneous moving frame for the given **t**, **tau** or **distance** or **velocity**.

t and **tau** always identify a unique point on the worldline. If a worldline maintains a velocity of 0 for some time, it will have multiple points with the same **distance** value. In this case, we choose the first matching point. Since worldlines have an infinite implied starting segment and since this segment could have a velocity of 0, this segment is skipped when searching for a matching distance—we only search through the explicitly defined segments. We use the same approach for **velocity**—ignoring any implied starting segment, we locate the first point with the matching velocity. If the worldline contains no such velocity, an error occurs.

The second form can be used to explicitly define the velocity and origin. If the velocity is omitted, it is set to 0. If the origin is omitted, it is set to (0, 0).

Frame objects contain two fields, named **v** and **origin**. These can be accessed or set using the "." operator.

Lines

line → **line** (
  (**axis** (**x** | **t**) *expr* (**offset** *expr)?*) |
  (**angle** *expr* **through** *expr*) |
  (**from** *expr* **to** *expr*)
)

In Gamma, lines are infinite, straight lines. They can be defined in three ways:

- Using a line parallel to the x or t axis of a given frame (the first expression) that crosses the axis not listed at the given offset (the second expression). For example, if we give the x axis, the offset represents a time value on the frame's t axis. If we give the t axis, the offset represents a position on the frames x axis. If the offset is missing, the offset is 0.
- Using an angle and a point through which the line goes. The angle is such that the x axis is 0° and the t axis is 90°.
- Using two points through which the line crosses (the line is still infinite).

Paths

path → *expr(coord)* (**,** *expr(coord)*)+

Unlike lines, paths are not infinite. They consist of a sequence of straight line segments, but may eventually include other curve types.

Paths can be used to draw areas and their borders. When used for areas, the last point in the path is always connected to the first point. A point is "inside" the area if a line drawn from that point to infinity crosses an odd number of the path segments that form the border.

Coordinates

coord → **(** *expr* **,** *expr* **)**

Coordinates are objects that don't follow the normal object syntax.

Coordinates form *(x, t)* pairs. Context determines of the values. For example, all drawing operations assume an *(x, t)* pair represents a location relative to the default frame.

Coordinate objects contain two fields, named **x** and **t**. These can be accessed or set using the "." operator.

Bounds

bounds → **bounds** *expr expr*

Bounds are also know as bounding boxes. The two expressions should evaluate to two coordinates. The coordinate values are sorted so that the smaller *x* and *t* values define the lower left corner of the bounding box and the larger *x* and *t* values define the upper-right corner. The coordinate values can be ±inf. Two -inf or two +inf values in the same range create a null bounds, one that will never include anything.

Intervals

interval → **interval** (**time** | **tau** | **distance**) *expr* **to** *expr*

An interval is a single range of *t*, *tau*, or *distance* values. The range values can be given in any order and a zero-length range is valid. The range can include ±inf. Two -inf or two +inf values create a null interval, one that will never include anything.

t and *tau* always identify a unique point in a worldline. *distance* does not: when both the acceleration and velocity of a worldline segment are 0, the first distance value will match the first point in a segment, as will the second distance value.

Variables

Variables are symbols which refer to a value. When referenced, the associated value is used in place of the variable.

The value's type is checked when a variable is referenced. If the required value is not of the required type or cannot be changed to the required type, an error is reported.

Array Variables

A variable name can be followed an expression enclosed in brackets. If the expression evaluates to a float, it is rounded towards 0 to an integer. The integer is converted to a string, prefixed with a dollar sign, and appended to the variable's name. If the expression evaluates to a string, the string is prefixed with a dollar sign and appended to the variable's name.

This creates a new name. Other than having the name created dynamically, the variable operates like any other variable. Note that it is impossible to create these variable names directly since the dollar sign is not valid in variable names. Note also that a[1] is equivalent to a["1"].

Predefined Variables

There are some predefined variables.

Numeric

- inf = positive infinity
- -inf = negative infinity

The infinity values are useful only in the few cases where a property value specifies infinity in the default, min, or max fields.

Boolean

- true = 1
- false = 0

Mathematics

- PI = the closest value to *pi*, the ratio of the circumference of a circle to its diameter.

Gamma

- E = The closest value to *e*, the base of the natural logarithms.

Static Variables

When a script is re-executed, the symbol table is cleared, removing all variables. a static variable, however, is retained from one execution to the next. It is created during the first script execution that defines it. It is used just like any other variable. However, during the next execution of the script, it continues to exist and retains its last assigned value.

Static variables would be useless if there were no way to detect if the variable already exists. Each execution would have to redefine it in exactly the same way and so the retained value would be overwritten. Attempting to use it without first creating it would yield an undefined variable error during the first execution.

The solution is a special function, defined(). The function takes one argument, a variable name, and it is the only place where an undefined variable does not yield an error. The function returns true if the variable is defined and false otherwise.

Dynamic Variables

Dynamic variables are special variables that can only be assigned to once and can only hold numeric values. Their values are controlled external to the script.

Animation variables are dynamic variables. If the **animation** command is used, their values depend on the animation frame being displayed.

Display variables are also dynamic variables. Their creation causes a GUI control to appear when the diagram is first displayed. Whenever the value of a display variable is changed using the GUI control, the script is re-executed and uses the new value.

See the section on the assignment statement for more detals.

Expressions

expr → *or* | (*expr* (**<-** | **->**) *or*)

or → *and* | (*expr* **||** *and*)

and → *equality* | (*expr* **&&** *equality*)

equality → *comparison* | (*expr* (**==** | **!=**) *comparison*)

comparison → *sum* | (*expr* (**<** | **>** | **<=** | **>=**) *sum*)

sum → *mult* | (*expr* (**+** | **-**) *mult*)

mult → *exp* | (*mult* (***** | **/** | **%**) *exp*)

exp → *unary* | (*unary* **^** *exp*)

unary → *postfix* | (**!** | **+** | **-**) *postfix*

postfix → *primary* | *postfix* **.** *name*

primary → *element* | **(** *expr* **)**

element → *float* | *variable* | *function* | *object*

function → *name* **(** *argList* **)**

argList → *expr* (**,** *expr*)*

Expressions are used to combine values into new values. Not all expressions are valid—syntactically valid expressions may be semantically invalid.

This table lists all the operators and the types of operands they operate on. Precedence can be controlled by using parentheses. Otherwise, the operations are performed in order of precedence (in the table, the lowest pre-

cedence numbers are the highest in precedence). With operators of equal precedence, operations are performed based on whether they are left- or right-associative.

Operator	Type	Precedence	Left Operand	Right Operand	Associativity	Result
.	*Binary*	1	*Object*	*Name*	Left	The value of the named field in the object.
!	*Unary*	2		*Any*	Right	For floats, a non-zero value becomes 0. Zero or null becomes 1. For all other values, a non-null value becomes 0 and a null become 1.
+	*Unary*	2	-	*Float*	Right	The value is unchanged.
-	*Unary*	2	-	*Float*	Right	The negated value.
^	*Binary*	3	*Float*	*Float*	Right	The left operand is raised to the power of the right operand.
*****	*Binary*	4	*Float*	*Float*	Left	The product of the two operands.
/	*Binary*	4	*Float*	*Float*	Left	The quotient of the two operands.
%	*Binary*	4	*Float*	*Float*	Left	The remainder of a division of the two operands.
+	*Binary*	5	*Float or String*	*Float or String*	Left	The sum of the two numbers or the concatenation of two strings. If one value is not a string and the other is a String, the value is converted to its String representation.
-	*Binary*	5	*Float*	*Float*	Left	The difference of the two operands
<	*Binary*	6	*Float*	*Float*	Left	1 if the left operand is less than the right; 0 otherwise.
>	*Binary*	6	*Float*	*Float*	Left	1 if the left operand is greater than the right; 0 otherwise.
<=	*Binary*	6	*Float*	*Float*	Left	1 if the left operand is less than or equal to the right; 0 otherwise.
>=	*Binary*	6	*Float*	*Float*	Left	1 if the left operand is greater than or equal to the right; 0 otherwise.
==	*Binary*	7	*Float*	*Float*	Left	1 if the left operand is equal to the right; 0 otherwise.
!=	*Binary*	7	*Float*	*Float*	Left	1 if the left operand is not equal to the right; 0 otherwise.
&&	*Binary*	8	*Float*	*Float*	Left	1 if both values are true. If the first value is fase, the result is 0 without evaluating the second value. A value is true if it is a float and non-zero and non-null or if it is anything else and not null; otherwise, it is false.
||	*Binary*	9	*Float*	*Float*	Left	1 if either value is true. If the first value is true, the result is 1 without evaluating the second value. A value is true if it is a float and non-zero and non-null or if it is anything else and not null; otherwise, it is false.
<-	*Binary*	10	*Coord*	*Frame*	Left	The inverse Lorentz transformation of the coordinate to the default frame.
->	*Binary*	10	*Coord*	*Frame*	Left	The Lorentz transformation of the coordinate to the given frame.

Functions

Unless stated otherwise, every function argument is an expression. The argument types listed below document the valid type accepted. Also unless otherwise stated, anywhere a *frame* is allowed, an *observer* may be used (the observer's frame at tau 0 is used).

Lorentz Transformations

Lorentz transformations of *(x, t)* from one inertial frame to another can be done using the **->** or **<-** operators, so a function for the transformation is not needed. The gamma value for a given velocity can be obtained using the

Gamma

gamma() function. The velocity can be given directly or extracted from a frame. (or indirectly, from an observer converted to a frame).

- **gamma(** *float* **)** — returns the gamma (γ) value for the inertial frame.

A simple inverse function exists. This only accepts a float representing a gamma value. The corresponding velocity is returned. The sign of the velocity will match the sign of the gamma value. A more precise function would accept only zero or positive values and return two velocity values, differing only by sign.

- **gammaToV(** *float* **)** — returns the velocity of an inertial frame with a given gamma (γ) value.

The following functions handle length contraction and time dilation. The first value is a proper length or duration measured in the frame given as the second parameter. The *frame* argument also accepts a velocity (relative to the rest frame) or an **observer**.

- **lengthContraction(** *float* **,** *frame* **)** — returns the contracted length as measured in the rest frame.
- **timeDilation(** *float* **,** *frame* **)** — returns the dilated duration as measured in the rest frame.

A velocity relative to one frame can be converted to the equivalent velocity relative to another frame. The first argument is a velocity. This velocity is relative to the rest frame unless the third parameter is given, in which case it is relative to that frame. The second argument is always the frame in which the final computed velocity is measured.

- **toRelativeV(** *float , frame* (**,** *frame*)* **)** — returns the velocity relative to the destination frame (second argument).

Angles

The angle of either spacetime axis (in degrees) can be determined from the velocity of the inertial frame associated with the axis. The velocity can be given directly or extracted from a frame. The angle returned is in degrees and is relative to the default frame's X axis, where 0° is parallel to the X axis and 90° is perpendicular. The value returned will always be between -90 (exclusive) and 90 (inclusive), although never ±45°.

- **toXAngle(** *float* **)**
- **toTAngle(** *float* **)**

So

```
let l = [line axis x frame1 offset 2];
```

is equivalent to

```
let l = [angle toXAngle(frame1) position (0, 2) <- frame1];
```

Use the following function to convert an angle relative to the rest frame to the equivalent angle in another frame. The first value is an angle in degrees relative to the rest frame. The second value can be a velocity (relative to the rest frame), frame, or observer. The result is the equivalent angle in the specified frame.

- **toRelativeAngle(** *float* **,** *frame* **)**

Worldline

As stated earlier, every point on a worldline has an associated set of scalar *x*, *t*, *d*, *tau*, and *v* values. The *t* and *tau* values are unique for each point. The *d* value either increments or stays the same. We can use the *d* value to uniquely identify the *first* point with that value. The x and v values are not unique and may occur in various disconnected regions.

The following functions are used to identify a given worldline point (using an *x*, *t*, *d*, *tau*, or *v* value) and return the other corresponding values. If the observer has an interval attached, only the portion of the worldline within the interval is searched. If there is no matching value, a null is returned. If there are multiple matching values (e.g. the velocity is constant and vToT() is called), the value corresponding to the earliest match is returned. This value could be -infinity.

- **dToT(** *float* **,** *observer* **)** — return *t*.
- **dToTau(** *float* **,** *observer* **)** — return *tau*.
- **dToV(** *float* **,** *observer* **)** — return *v*.

- **dToX(** *float* **,** *observer* **)** — return *x*.
- **tToD(** *float* **,** *observer* **)** — return *d*.
- **tToTau(** *float* **,** *observer* **)** — return *tau*.
- **tToV(** *float* **,** *observer* **)** — return *v*.
- **tToX(** *float* **,** *observer* **)** — return *x*.
- **tauToD(** *float* **,** *observer* **)** — return *d*.
- **tauToT(** *float* **,** *observer* **)** — return *t*.
- **tauToV(** *float* **,** *observer* **)** — return *v*.
- **tauToX(** *float* **,** *observer* **)** — return *x*.
- **vToD(** *float* **,** *observer* **)** — return *d*.
- **vToT(** *float* **,** *observer* **)** — return *t*.
- **vToTau(** *float* **,** *observer* **)** — return *tau*.
- **vToX(** *float* **,** *observer* **)** — return *x*.

Bounds

A single bound can be attached to a line. The line (which is normally infinite) will clip the line to lie within the box. This can effectively turn a line into a line segment or delete it completely.

Bounds are used for intersections and when drawing a line.

- **setBounds(** *line* **,** *bounds*) — returns a bounded line.
- **clearBounds(** *line* **)** — returns an unbounded line.

Intervals

A single interval can be attached to an observer. This produces an interval-limited observer. Setting an interval removes any existing interval, so only one interval is present at any time. The clearInterval() function can be used to remove any interval.

Intervals are used in worldline and intersection functions, and when drawing a worldline.

- **setInterval(** *observer* **,** *interval* **)** — returns an observer.
- **clearInterval(** *observer* **)** — returns an observer.

Intersections Functions

These functions return a coordinate representing the intersection of two lines, or a line and a worldline, or two worldlines.

- **intersect(** *line* **,** *line* **)**
- **intersect(** *line* **,** *observer* **)**
- **intersect(** *observer* **,** *line* **)**
- **intersect(** *observer* **,** *observer* **)** *— Not implemented.*

When intersecting a line with a line, the results are always a single point unless:

- The lines are the same, in which case they intersect at every point.
- The lines are parallel, in which case (except for the above) they don't intersect.
- The intersection falls outside of a line's bounding box.

Any of these conditions returns a null.

When intersecting a line with an observer, we intersect the line with each worldline segment, from earliest to latest, until we find an intersection, which is returned. If the intersection consists of multiple points (e.g. one line on top of another), the earliest point in the intersection is returned. If there are no intersections, a null is returned.

Gamma

When intersecting two observers, the first segment of the first worldline is checked against all the segments of the second (from earlier to later), then the second segment, and so on. If the intersection consists of multiple points, the earliest point in the intersection is returned. If there are no intersections, a null is returned.

If a line is bounded, the intersection must fall within the bounds. If a worldline has an associated interval, the intersection must fall within the interval. Failure in either case causes a null to be returned.

Basic Math Functions

These perform their typical mathematical operation. For a precise definition for most of these, refer to the Java Math package.

- **abs(** *float* **)** — returns a floating point number.
- **acos(** *float* **)** — returns a floating point number.
- **acosh(** *float* **)** — returns a floating point number.
- **asin(** *float* **)** — returns a floating point number.
- **asinh(** *float* **)** — returns a floating point number.
- **atan(** *float* **)** — returns a floating point number.
- **atan2(** *float* **,** *float* **)** — returns a floating point number.
- **atanh(** *float* **)** — returns a floating point number.
- **ceil(** *float* **)** — returns a floating point number.
- **cos(** *float* **)** — returns a floating point number.
- **cosh(** *float* **)** — returns a floating point number.
- **exp(** *float* **)** — returns a floating point number.
- **floor(** *float* **)** — returns a floating point number.
- **log(** *float* **)** — returns a floating point number.
- **log10(** *float* **)** — returns a floating point number.
- **max(** *float* **,** *float* **)** — returns a floating point number.
- **min(** *float* **,** *float* **)** — returns a floating point number.
- **random( )** — returns a floating point number.
- **round(** *float* **)** — returns a floating point number.
- **sign(** *float* **)** — returns a floating point number.
- **sin(** *float* **)** — returns a floating point number.
- **sinh(** *float* **)** — returns a floating point number.
- **sqrt(** *float* **)** — returns a floating point number.
- **tan(** *float* **)** — returns a floating point number.
- **tanh(** *float* **)** — returns a floating point number.

Wavelength and Frequency Functions

The next four functions deal with an electromagnetic signal that travels between two observers. Each observer notes the frequency or wavelength of the signal as it passes by.

The first two functions take the two measurements and compute the velocity of the first observer as measured by second observer relative to the comoving frame of the second observer at the instant this observer measures the signal. The units used to measure the wavelengths or frequencies don't matter as long as they are consistent.

- **dopplerWavelengthToV(** *float* **,** *float* **)** — returns a velocity.
- **dopplerFrequencyToV(** *float* **,** *float* **)** — returns a velocity.

The next two functions invert the above. The first argument is the wavelength or frequency as measured by the first observer. The second is the velocity of the first observer measured by the second observer relative to the comoving frame of the second observer at the instant this observer measures the signal.

- **dopplerVToWavelength(** *float* **,** *float* **)**
- **dopplerVToFrequency(** *float* **,** *float* **)**

The next two functions convert electromagnetic signal frequencies (in Hz) to wavelengths (in meters) and vice versa.

- **frequencyToWavelength(** *float* **)**
- **wavelengthToFrequency(** *float* **)**

String Functions

- **toString(** *float* **,** *float* **)**

Convert the first numeric expression to its character equivalent. The second argument gives the number of digits to the right of the decimal point. This function ignores the display and print precision values set in the **set** statement.

Definition Functions

- **isDefined(** *variable* **)** — returns 1 (true) if the variable is defined.

The function works with array variables, which are just a different way of naming a variable.

Slideshows

slideshow → **slideshow** *slideshowSettings*? *slideshowBlock*

slideshowSettings → *slideshowSetting* (**,** *slideshowSetting*)*

slideshowSetting → **autoPlay** | **defaultPause** *float*

slideshowBlock → **{** *scriptStatement* (**;** *scriptStatement*)* **}**

scriptStatement → **script** *string scriptSettings*?

scriptSettings → *scriptSetting* (**,** *scriptSetting*)*

scriptSetting → **pause** *float*

A script whose first token is **slideshow** is a special type of script, one composed of other scripts meant to be run in a sequence. When a script begins with slideshow, it is processed entirely by the parser; therefore, all values must be literals—neither variables nor expressions are allowed.

The slideshow script contains a single block comprised of external scripts. These are identified by a relative or absolute filename or URL. If relative, the external script nane is relative to the slideshow file or URL.

If the external script is itself a slideshow, the scripts in the external slideshow are added to the current slideshow as though they had been entered in place of the external script.

A slideshow has two optional arguments. These can appear in any order, but only once:

- **autoRun**: If provided, the slideshow begins in the "running" state. Otherwise, it begins in the "paused" state.
- **defaultPause**: This is the value to pause for any script without an explicit pause statement. The default is to pause for 5 seconds.

These settings are ignored for any embedded script which is itself a slideshow. Only the outermost slideshow settings are used.

Each individual script may be followed by a **pause** setting, which specifies a pause time in seconds. Gamma will execute the next script in the slideshow after:

Gamma

- The diagram is drawn or, if animated, the last frame is drawn AND
- There is no end user interaction with the diagram within the pause time frame

End user interactions include:

- Zooming and panning
- Manipulating a GUI control created using display variables
- Moving the animation to a frame other than the last one

If a slideshow is manually paused by the user, the current script continues running until the user ends the pause, at which time the next slide is immediately executed. If the current slide is the final slide, pause and run have no effect.

The final slide is executed until the user moves to a prior slide or to the first slide, or exits the program.

Slideshows cannot include any statements other than **slideshow** and **script**.

Include Statement

include *string*;

Some collection of commands might be reusable across multiple scripts, so there is a provision for incorporating a file into the main script.

The **include** statement is processed by the parser and must use a literal string. The string can be an absolute or relative filename or URL. If relative, it is relative to the main script file or URL. The effect is equivalent to replacing the include statement with the text in the file.

If the current script came from an URL, the **include** URL must be from the same domain. If the current script came from a file, the URL can be from any domain, but will not be monitored for changes.

The include statement can be used anywhere a statement is allowed.

Stylesheet Statement

stylesheetStatement → **stylesheet external*** *string*

If **external** is specified, the string contains an absolute or relative filename or URL. If relative, it is relative to the main script file or URL. The filename should contain stylesheet rules, following the syntax for stylesheets.

If the current script came from an URL, the **stylesheet** URL must be from the same domain. If the current script came from a file, the URL can be from any domain, but will not be monitored for changes.

If **external** is omitted, the string contains a stylesheet. This can be awkward since stylesheets can contain strings and so any embedded quotes must be properly escaped.

Like the include statement, the **stylesheet** statement is processed by the parser and must use a literal string. This restriction is mostly for performance, as it allows the styles to be completely determined before execution begins.

Set Statement

setStatement → **set** *setting* (**,** *setting*)*

setting → **units :** *float* | **displayPrecision :** *float* | **printPrecision :** *float* | **minVersion :** *string*

The **set** statement is another parser command and thus all values must be literals.

The **units** clause defines the meaning of one unit in the diagram. A value of 1 means that units are in light years, x, and years, t. The choice of units affects the interpretation of acceleration values (which are in g's). If omitted, the value used is 1.

The **displayPrecision** clause sets the default precision for any number that is converted to a string and displayed on the diagram. The value represents the number of digits to the right of the decimal point. The **toString()** function is the only way to override this. If omitted, the default value is 12.

The **printPrecision** clause sets the default precision for any number that is converted to a string and printed with the **print** statement. The value represents the number of digits to the right of the decimal point. The **toString()** function is the only way to override this. If omitted, the default value is 12.

The **minVersion** clause defines the minimum acceptable version of Gamma for the script. It is given as a string in the form **"***digit+***.***digit+***.***digit+***"**. If the current version of Gamma is lower than the given version, the script will not be executed and an explanatory message will appear with a link to the latest release.

Within one **set** statement, only one clause of each type can be entered. If multiple **set** statements are used, the settings are merged, with the last value set of a given type winning. The final values set apply to the entire script, not just to the statements that come after it.

Print Statement

printStatement → **print** *expr*

The first occurrence of a print statement causes a console dialog to appear. The dialog will contain a string representation of the expression, followed by a newline. Every additional print statement adds to the output.

Numbers printed directly or as part of an object will have their significant digits constrained to the number given in the **printPrecision** clause of the **set** statement.

If Statement

ifStatement → **if (** *expr* **)** *statement* (**else** *statement*)?

For an **if** statement, the statement that follows the **if** is executed if the expression evaluates to a non-zero value. If it evaluates to 0, then the statement following the **else** is executed, if present.

While Statement

whileStatement → **while (** *expr* **)** *statement*

If the expression evaluates to a non-zero value, the statement is executed. The sequence repeats until the expression evalutes to 0 (but see the **continue** and **break** statements).

For Statement

forStatement → **for** *variable* **=** *expr* **to** *expr* **step** *expr statement*

The variable is created if it doesn't exist and is then assigned the value of the first expression, which must be numeric. The variable is then checked to see if it is greater than the second expression (or less than, if the **step** expression is negative). If this test passes, the statement is executed. The variable is then automatically incremented by the step value and the sequence repeats from the test until the test fails (but see the **continue** and **break** statements).

After exiting the **for** loop, the variable retains its last assigned value, the value that exceeded the limit (but see the **break** statement).

Continue Statement

continueStatement → **continue**

The **continue** statement returns to the test portion of a **while** statement or to the increment and test portion of a **for** statement.

Break Statement

breakStatement → **break**

The **break** statement exits a **while** or **for** loop (it jumps to the first line of code after the loop. The **for** loop variable is not increment further.

Assignment Statement

assignmentStatement →
(*leftVariable* **=** *expr*) |
(**animate** *leftVariable* **=** *expr* (**to** *expr*)? **step** *expr*) |
(**range** *leftVariable* **=** *expr* **from** *expr* **to** *expr* **label** *expr*) |
(**toggle** *leftVariable* **=** *expr* **label** *expr* **restart**?) |
(**choice** *leftVariable* **=** *expr* **choices** *expr* (**,** *expr*)* **label** *expr* **restart**?)

Assignment statements assign a value to a variable. If the variable doesn't yet exist, it is created.

The assignment statement creates a variable if it doesn’t yet exist and stores the associated value in the variable’s storage location, overwriting anything previously there. As there are different types of values, an update may also change the variable’s type.

There are several types of assigments. The main type (the one that doesn't start with a keyword) is used to assign expressions of any type to a variable, overwriting whatever value may be currently stored by that variable.

The other types of assignments are used to create *dynamic variables*, variables whose values can be changed from outside the script. Dynamic variables can only store numeric values and can only be assigned to once. The assigned value (the value after the equals sign) is called the initial value. It is used the first time a script is executed and may be changed externally after that.

There are two types of dynamic variables: *animation variables* and *display variables*.

Animation variables are created with the **animate** keyword and are used to animate a diagram. The value of each animation variable is incremented by its step size until its limit is exceeded, if one is specified. The first animation variable to exceed its limit ends a single repetition of an animation. If no animation variable has a limit, then the animation is theoretically infinite, although in practice there is a finite (but large) limit.

If the step size is positive, the limit must be greater than the initial value and the limit is exceeded when the incremented value is greater than the limit. If the step size is negative, the limit must be less than the initial value and the limit is exceeded when the incremented value is less than the limit.

Animations only run when the **animation** command is present and there is at least one animation variable; otherwise, they operate like any other variable.

A display variable causes a GUI control to appear. There are three types:

- When created with the **range** keyword, a slider is created. The **from** and **to** values represent the range of the slider. These are automatically sorted so that the from value is always less than the to value. The initial value must fall inside the range (inclusive). When the user manipulates the slider, the script is re-executed with the slider's new value used as the value of the range variable.
- When created with the **toggle** keyword, a checkbox is created. If the initial value is non-zero, the checkbox is enabled (checked). When the user enables the checkbox, the corresponding variable's value is set to 1.0.

When the checkbox is disabled, the value is set to 0.0. If the **restart** option is included, any running animation is restarted—this may be necessary if an animation variable's range depends on the toggle's value.

- When created with the **choice** keyword, a drop-down list is created. The expressions in the **choice** clause must evaluate to strings and will be displayed as the choices in the list. The initial value is converted to an integer and represents the initial choice, with 1 selecting the first choice item. The initial value must be in the range from 1 to the number of choices (inclusive). When the user selects a new choice, the variable's value changes to the number of the choice. If the **restart** option is included, any running animation is restarted—this may be necessary if an animation variable's range depends on the choice.

All display variables have a **label**. The value of a label can be anything, although it is probably best to use a string. Other values are converted to strings. The label is displayed with the GUI control.

Display variables are added to the window in the order in which they are created in the script.

Commands

commandStatement → *command propertyList*?

Command statements are statements that control the diagram (**display**, **animation**, and **frame**) or cause things to be drawn (everything else).

Property Lists and Properties

propertyList → *propertyElement,* (**,** *propertyElement*)*

propertyElement → *property* | *expr*

property → *name* **:** *expr*

Commands may have a large number of settings. Many settings are optional and have a default value if omitted. Rather than include every setting, one can include only the required settings and the settings with non-default values.

A property identifies a setting and assigns it a value. A property list assigns values to a list of properties.

In this document, properties will be listed in tables. The table lists the property name, the type of value it accepts, its default value if omitted, and its minimum and maximum values, if any. Properties of the wrong type or outside the legal range will generate errors."solid"In the following list,

- If a float, int, color, boolean, or angle is required, an expression that evaluates to a float is accepted.
- If a string is required, any expression that evaluates to a float or string is accepted.
- If a frame is required, any expression that evaluates to either an observer or a frame is accepted.
- Otherwise, the expression must evaluate to the named type.

All commands include three special properties: **id**, **class**, and **style**. The values of each of these should be a string. The use of these properties is described in the Stylesheet Semantics section.

The following rules are used in processing a property list:

- If the same property name appears more than once, the last one wins.
- If any property is included that does not belong to the given command, an error occurs.
- If any required property (one with no default value) is omitted, an error occurs.
- Any non-required properties that are omitted are given their default value.

Default and Required Properties

In the following lists, some properties can be given without the property name or colon. In these cases, they must be the first value following the command name. If a property is required, it will be listed in red. If it is optional, it will be listed in green.

Gamma

The effect is the same as if the property had been given with preceded by the name. Required properties must be given without the property name as the first value, even if they later appear with the property name.

display

The display command is special in that it is executed before any other command, regardless of its location in the script. If more than one display command is given, the last one is used; the others are completely ignored. If none is given, one is created which assigns the default values to each property.

The display command cannot be animated. It sets up the diagram size, the way the diagram reacts to window size changes, the units, and the initial zoom and pan. Animating most of these would usually be detrimental. It would be useful to animate the zoom and pan, but it would interfere with the user's ability to zoom and pan.

A future version might allow animating zoom and pan perhaps by disabling the user's ability to zoom and pan.

If an animation is running, then only the first set of property values are used.

Name	Type	Default	Min	Max	Description
width	*int*	*int*	1	Maximum display width	The width of the diagram in pixels. If both **width** and **height** are omitted, the display area is always resized to the maximum space available. If only **width** is omitted, it is set to the maximum window width, but is not resized after that.
height	*int*	*int*	1	Maximum display height	The height of the diagram in pixels. If both **width** and **height** are omitted, the display area is always resized to the maximum window size. If only **height** is omitted, it is set to the maximum space available, but is not resized after that.
units	*string*	pixels			The units for width and height. Valid values are "pixels", "inches", or "mm". The size of the image on the display will be correct for the units chosen only on the monitor on which the image is created. If moved to another monitor, the operating system may choose to retain the physical size at the expense of the pixel size. When the units are in pixels, the physical size of the image will vary on different monitors; if the units are in inches or millimeters, the size should be consistent, but the number of pixels will vary.
origin	*coord*	(50, 50)			The location of the origin of the default observer on the diagram. The display width is divided into 100 units such that 0 represents the left side of the diagram and 100 represents the right. The display height is also divided into 100 units where 0 is the bottom and 100 is the top of the diagram. Values are not limited to being from 0 to 100, though—the origin can be off-screen.
scale	*float*	1	>0		Unscaled coordinates (i.e. pixels) are scaled by this factor to produce scaled coordinates. The scale is defined by taking the lesser of the diagram width or height. A scale of 1 means that this distance is exactly 100 units, a scale of 2 would be 50 units, a scale of .5 would be 200 units and so on.

origin and **scale** are used when the diagram is first drawn. They establish the zoom and pan and can be changed when the user zooms and pans.

frame

The frame command defines the drawing frame. Like the diagram command, it is also special in that it is executed before any other command, regardless of its location in the script. If more than one display command is given, the last one is used; the others are completely ignored. If none is given, one is created which assigns the default values to each property (basically, it sets the drawing frame equal to the default frame).

The frame command *can* be animated.

Name	Type	Default	Min	Max	Description
frame	*frame*	defFrame			The inertial frame from whose perspective the diagram will be drawn.

animation

The animation command is evaluated in place but applied once the script is executed but before display begins. It signals that the diagram should be animated. If more than one animation command is given, the last one is used; the others are completely ignored. The animation command cannot itself be animated; only the first set of property values are used.

Name	Type	Default	Min	Max	Description
cycle	*boolean*	false			If **cycle** is false, one repetition of the animation is completed once the first animation variable with a limit exceeds its limit. If **cycle** is true, one repetition of the animation is completed once the first animation variable with a limit exceeds its limit and the frames are executed in reverse order.
reps	*int*	500	1	500	The number of repetitions of the animation.
speed	*float*	1	0.1	10	A multiplier applied to the "normal" speed (a value of 1 is "normal" speed, a value of 2 is twice as fast, and a value of 0.5 is half as fast).

To animate a diagram, at least one animation variable needs to be defined and the animation command must be included; otherwise, there is no animation.

The animation algorithm is as follows:

- Set the repetition counter to 1.
- Set a animation frame number counter to 1.
- Execute the script. When an animation variable assignment is encountered, create and initialize the variable. Compute and store its limit (if any) and step values.
- Report an error if an animation variable appears on the left side of more than one assignment statement.
- Increment frame counter by one and then increment every animation variable by its step size.
- If none of the animation variables exceeds its limit (for positive step values) or is less than its limit (for negative step values), generate the next frame by executing the script again, ignoring any **animation** statements and animation assignments.
- Repeat the above step until at least one animation variable fails its limit test. This may never happen and the animation may need to be stopped externally. If one animation variable fails, then we will have generated frames 1 through n (where n could be 1).
- If **cycle** is true and there are more than 3 frames, produce the frames in reverse. This set of frames will be exactly the same as frames n-1 through frame 2, with the frame counter decrementing by 1.
- Increment the repetition counter by 1. If the repetition counter is equal to the **reps** value, then:
  - If **cycle** is false, the animation is complete.
  - If **cycle** is true, execute frame 1 again so that we finish exactly where we started.

During animation, controls for pausing, resuming, and single-stepping forward and backward will be displayed.

The goal is to animate at 30 frames per second. An animation with 30 frames should last one second when played at normal speed.

axes

Draw the axes for a given inertial frame.

Gamma

Name	Type	Default	Min	Max	Description
frame	*frame*	defFrame			The frame that defines the axes.
positiveOnly	*boolean*	false			Draw only the positive axes; otherwise, draw the full axes.
x	*boolean*	true			Draw the x axis.
t	*boolean*	true			Draw the t axis.
xLabel	*string*	""			The label for the x axis.
tLabel	*string*	""			The label for the t axis.
style					The axes and text style properties are used.

The x and t axes labels will be placed depending on the scale and pan.

- The x label is placed at the center point on the larger of the $-x/+x$ segment visible. The t label is handled similarly.
- The labels are drawn upright (0° rotation). The **textRotation** style is ignored.
- The labels are drawn on the side opposite the tick labels. If the tick labels aren't displayed, they are displayed on the same side as the tick labels would have been.
- The text anchor is chosen automatically. The **textAnchor** style is ignored.
- The label is not drawn if the axis is not visible.

The axes labels are in the style of map labels on interactive maps—these labels may appear, disappear, or get repositioned as one scales and pans the map. The **label** command can be used to create labels whose positions remain fixed; however, even then, the text size will not scale when zooming in or out.

grid

Draw the grid for the given inertial frame. The grid density will be automatically adjusted based on scaling and the angle of the frame axes.

Name	Type	Default	Min	Max	Description
frame	*frame*	defFrame			The frame that defines the axes.
x	*boolean*	true			Draw the lines parallel to the x axis.
t	*boolean*	true			Draw the lines parallel to the t axis.
style					The line style properties are used.

hypergrid

Draw a hyperbolic grid. The grid density will be automatically adjusted based on scaling.

Name	Type	Default	Min	Max	Description
style					The line style properties are used.

event

Draw an event (represented by the selected event shape) at a particular location or draw all instances of an event as it appears in all frames between two frames.

The location is always given relative to the default frame (the coordinate syntax allows entering coordinates relative to any frame).

Name	Type	Default	Min	Max	Description
location	*coord*	none			The event location.
text	*String*	""			A label for the text.

Name	Type	Default	Min	Max	Description
rotation	*angle*	0.0			The rotation of the text in degrees around the anchor point. Positive rotation values rotate counter-clockwise. 0 is horizontal.
frameRelative–Rotation	*boolean*	false			If true, the rotation angle changes with the drawing frame. If false, it's frame-independent.
boostTo	*frame*	none			The frame to boost to. The location is boosted to this frame and a hyperbolic line joins the two positions. If boostTo is omitted, the selected event shape is drawn. If a frame is supplied, a hyperbolic line connnects the location relative to the default frame and the location relative to the boost frame.
style					The event and text properties are used.

line

Given a line, this command draws the line or a portion of the line. If the line has a bounding box attached, only the portion within the bounding box will be drawn. Non-infinite ends of a line may be given arrowheads.

Name	Type	Default	Min	Max	Description
line	*line*	none			The line to draw.
style					The line properties are used.

worldLine

Given an observer, draw the observer's worldline or a portion of the worldline. If the worldline has an interval attached, only the portion within the interval will be drawn. Non-infinite ends of a worldline may be given arrowheads.

Name	Type	Default	Min	Max	Description
observer	*observer*	none			The observer whose worldline will be drawn.
style					The line properties are used.

path

Given a path, draw the path and/or fill the area it contains.

If **closed** is true, the last point in the path is treated as connected to the first point.

If **stroke** is true, the path is drawn using all the styles relevant to lines.

If **fill** is true, the path is treated as closed and filled using all the styles relevant to fills.

Name	Type	Default	Min	Max	Description
path	*path*	none			The path to use in creating the fill.
closed	*boolean*	false			Whether to close the path by connecting the first point to the last.
stroke	*boolean*	true			Whether to draw the path.
fill	*boolean*	false			Whether to fill the path. For the fill, the path is treated as closed
style					The fill properties are used.

label

Draw text. The size of the bounding box of the text string is determined, padding is added, and the attachment point is selected. The text is placed with its attachment point at the given location and is then rotated the given amount.

Gamma

Name	Type	Default	Min	Max	Description
location	*coord*	none			The position to draw the text at.
text	*string*	""			The text to draw.
rotation	*angle*	0.0			The rotation of the text in degrees around the anchor point. Positive rotation values rotate counter-clockwise.
frameRelative–Rotation	*boolean*	false			If true, the rotation angle changes with the drawing frame. If false, it's frame-independent.
style					The text properties are used.

User Interface

Scripts

Scripts are edited using any text editor. When the file is saved in the text editor, the diagram is updated.

A future version of Gamma may support an internal editor. The script text would then be edited in a separate window and executed when the script is saved. Until a script is saved, a relative include statement will generate an error.

Command Line

The command line will accept any number of space-separated file names. Each existing file will be associated with a new Gamma window.

The command line will also accept **-h** or **-help** to display a short help message for the command line options, and **-v** or **-version** to display the version number.

If **-h** or **-help** is given, all other options are ignored. Otherwise, if **-v** or **-version** is given, all other options are ignored.

The Menu Bar

File Menu

- **New...**: Displays a dialog asking for the name of a new script file. The file directory displayed will be *the default directory for scripts*. If the file exists, an error will be reported. The file can have any extension.
- **Open...**: Displays a dialog asking for the name of a existing script file. The file directory displayed will be *the default directory for scripts*.
- **Open URL...**: Displays a dialog asking for a URL to open. If the protocol is missing, "http://" is added. The file read will be parsed as a script. Any dependent files must either use relative links or come from the same domain as the script. If Gamma is set up to open script files in an editor, this step is skipped.
- **Export Diagram...**: Displays a dialog asking for the name of a file in which to save the diagram. The file directory displayed will be *the default directory for images*. If the file exists, a confirmation dialog will appear. This option is only enabled when a script file has executed without errors.

  The size of the saved image will be the same as the current image size. If an animation is running, it will be paused and the saved image will match the paused diagram.

  The last format choice is retained for the next export, even across executions of the program. Images can be saved in Jpeg and PNG formats. Format-specific settings are retained for the next use of the format, even across executions of the program.
- **Export Video...**: Displays a dialog asking for the name of a file in which to save an animation video. The file directory displayed will be *the default directory for videos*. If the file exists, a confirmation dialog will appear. This option is enabled only when an animated diagram is present has displayed at least one frame without errors. If an execution error occurs will exporting a video, the video will contain only the frames which executed without errors.

  The size of the saved image will be specified in the export dialog. The dialog will display the total length of the video (including repetitions, loops and cycles). A shorter time may be entered. The animation will be saved from the start but using the current display variable settings.

Gamma

The last format choice is retained for the next export, even across executions of the program. Images can be saved in Jpeg and PNG formats. Format-specific settings are retained for the next use of the format, even across executions of the program. (Not yet implemented.)

- **Print...**: Displays a print dialog with the usual options. This option is only enabled when a script file has executed without errors. (Not yet implemented.)

  The image can be printed in portrait mode, landscape mode, or best fit, based on the current diagram size. The image can be scaled to fit or to specific dimensions, although the aspect ration will be maintained. If the image falls outside the boundaries of the print area, a confirmation dialog will appear before printing.

  The default printer is the initial printing choice. If changed, the new printer becomes the choice for further printing, until the printer is changed again. The orientation and scaling settings are also retained until the end of the program.
- **Close Window**: Closes the current window.
- **Preferences...**: Displays the preferences dialog. Saved preferenes include the following:
  - Whether the greeting message should be displayed.
  - The default directory for scripts: If not set, the directory Gamma/Scripts is used in the user's home directory.
  - The default directory for images: If not set, the directory Gamma/Images is used in the user's home directory.
  - The default directory for videos: If not set, the directory Gamma/Videos is used in the user's home directory.
  - A command to use to open script files. If omitted, no attempt is made to open the script.
  - A default user stylesheet to use for all scripts. If omitted, this stylesheet is skipped.
  - The last selected export image format. This is part of the preferences, but is not included in the Preferences dialog. If not set, the option is PNG.
  - The last specified options for each export image format. This is part of the preferences, but is not included in the Preferences dialog.
  - The last selected export video format. This is part of the preferences, but is not included in the Preferences dialog. If not set, the option is ???.
  - The last specified options for each export video format. This is part of the preferences, but is not included in the Preferences dialog.

Window Menu

- **New Window**: Creates a new Gamma window with no associated file.

Help Menu

- **Contents**: Displays help content.
- **About Gamma**: Displays an About dialog that includes the version number.

Default Directories

The directory used for open/new/export dialogs for each type of file (script, image, video) is chosen this way:

- If the program has just started and a window is not associated with a file, the directory is the default directory for that file type.
- If the program has just started and a window is associated with a file, then for script files only, the directory is the same directory as for the associated file.

- Once the user selects a file from a different directory, that directory becomes the default for that file type, but only for that window.
- If a window is created from the Window/New Window menu, it inherits the the directories for each file type from its parent window.

The Toolbar

The toolbar will contain icons for the following menu functions

- File / New
- File / Open
- File / Open URL
- File / Export Diagram

It also contains icons for the following generic functions:

- Re-load the current script—this restarts the script as though it had been freshly opened. If a slideshow is running, this reloads the current slide, not the slideshow.

And for the following slideshow functions:

- First script (disabled when running the first script)
- Last script (disabled when running the last script)
- Previous script (disabled when running the first script)
- Next script (disabled when running the last script)
- Play/Pause slideshow (disabled when running the last script)

The slide show icons are enabled only when a slideshow is loaded.

Slideshows

When the first/last and previous/next buttons are used, the current slide is immediately terminated and a new slide is executed.

When a Play/Pause is on Play, slides advance automatically, following rules stated earlier. When Play/Pause transitions to Pause, the current slide runs indefinitely. When Play/Pause transitions to Play, the current slide is immediately terminated and the next slide is executed.

Diagrams

When a script is executed, the **display** command sets the size of the diagram. This may be a fixed size or it may to set the diagram size to the size of the drawing area of the window. If the size is fixed, it may be bigger or smaller than the drawing area—if bigger, some of the diagram will be clipped; if smaller, the diagram will be centered in the drawing area.

Animation Controls

Animation controls appear in the right side of the status bar. These are disabled unless an animation is executed.

- The left-most button moves to the first frame of the animation. This is disabled when on the first frame and when the animation is running.
- The next button allows single-stepping to the previous frame. This is disabled when on the first frame and when the animation is running.

Gamma

- The next button toggles between play and stop. If the animation is running, it will stop it on the current frame. If the animation is stopped, it will continue it from the current frame. If the animation is at the end, it will be disabled.
- The next button allows single-stepping to the next frame. This is disabled when on the last frame and when the animation is running.
- The right-most button moves to the last frame of the animation. This is disabled when on the last frame and when the animation is running.

The full length of the animation depends on the choice of loop or cycle, the number of repetitions, and whether any animation variable has a limit. Moving to the start, or end, or moving forward and backward one step all work based on the total animation length rather than on just the length of one loop.

The animation can also be controlled from the keyboard:

- The space key toggles between start/resume if the animation is stopped and pause if the animation is running.
- The escape key stops the animation if it is running. Otherwise, it has no effect.
- The left arrow key steps one frame backward if the animation is paused and there is at least one more frame preceding the current frame. Otherwise, it does nothing.
- The right arrow key steps one frame forward if the animation is paused and there is at least one more frame following the current frame. Otherwise, it does nothing.
- The up arrow key speeds up the animation.
- The number 1 sets the animation speed to 1. This might not be the speed set as the default speed in the script.
- The down arrow key slows down the animation.
- Left brace goes to the first frame and stop.
- Right brace goes to the last frame and stops.

Zoom and Pan

- Ctrl++ zooms in
- Ctrl+- zooms out
- The mouse scroll wheel can zoom in and out.
- Left mouse drag pans.
- Ctrl+0 resets the zoom and pan to match the initial zoom/pan.

Zoom and pan work during animations. Zooming with the scroll wheel always keeps the point under the mouse cursor in the same position. Zooming with the keyboard keeps the point at the center of the diagram in the same position.

Cursor Feedback

The rest coordinates of the position under the mouse cursor are always displayed in the status bar.

Console Printing

The **print** statement's output is displayed on a console dialog that appears only if something is printed. The contents of the dialog can be printed to a printer.